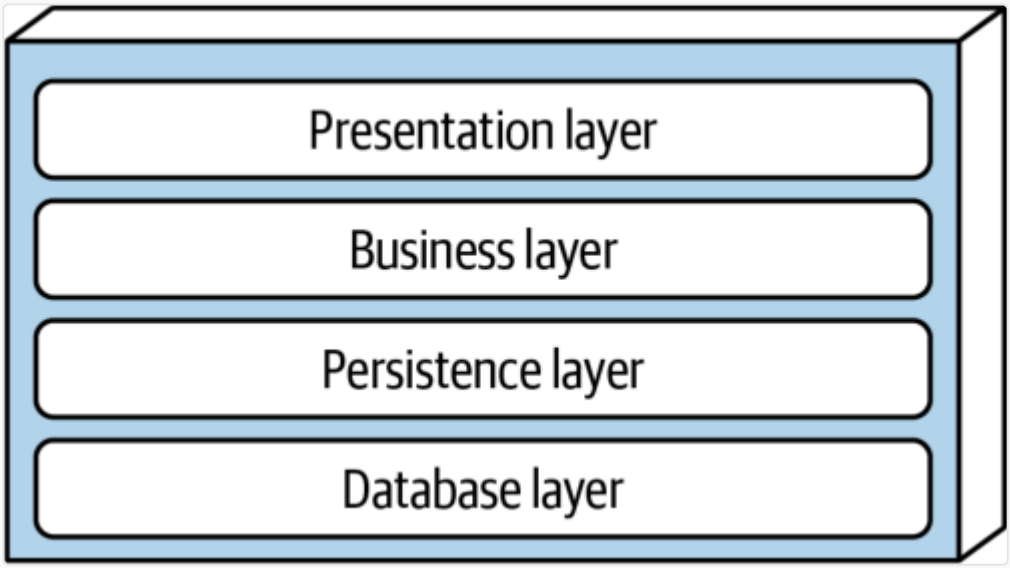
فصل ۱۰: سبک معماری لایه‌ای

سبک معماری لایه‌ای، که معماری چندلایه `(n-tiered)` نیز نامیده می‌شود، یکی از رایج‌ترین سبک‌های معماری است. این سبک به دلیلِ سادگی، آشنایی و هزینهٔ پایینش، استانداردِ عملیِ بسیاری از برنامه‌های میراثی است.

سبک معماری لایه‌ای ممکن است به دامِ چند ضدالگوی معماری بیفتد، از جمله ضدالگوهای معماری ضمنی `(Architecture by Implication)` و معماری اتفاقی `(Accidental Architecture)`. وقتی توسعه‌دهندگان یا معماران «فقط شروع به کدنویسی می‌کنند» و مطمئن نیستند از کدام سبک معماری استفاده می‌کنند، احتمالِ زیادی دارد که در حالِ پیاده‌سازیِ سبک معماری لایه‌ای باشند.

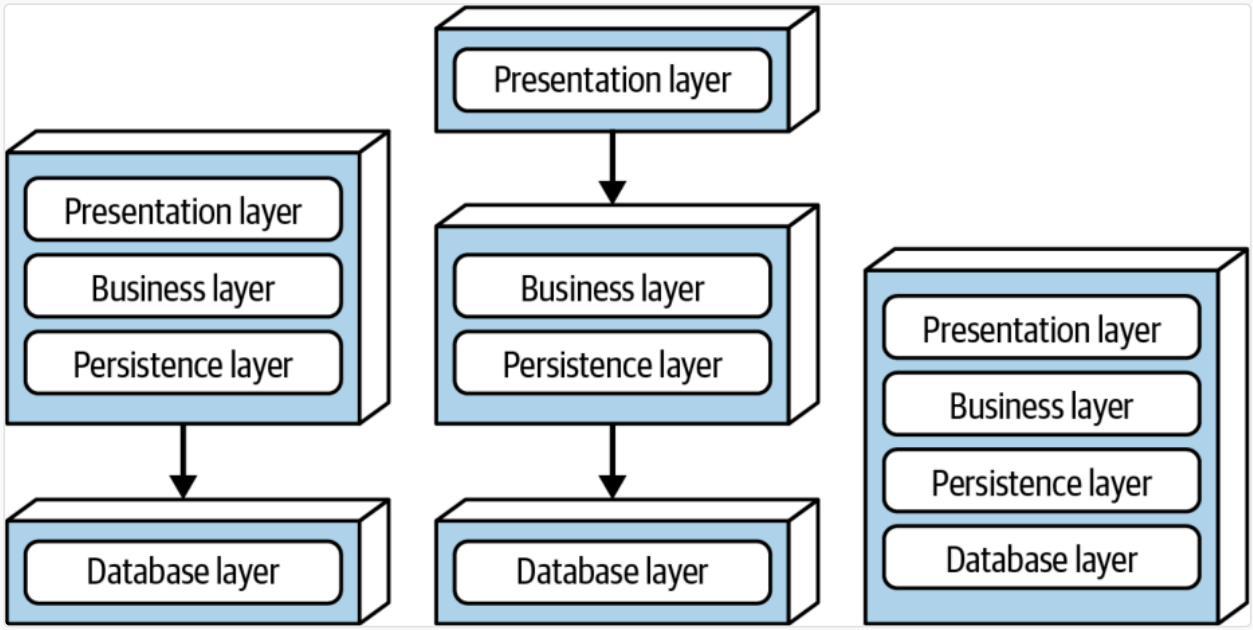
توپولوژی

مؤلفه‌ها در سبک معماری لایه‌ای در لایه‌های افقیِ منطقی سازمان‌دهی می‌شوند، چنان‌که در شکل ۱۰-۱ نشان داده شده است؛ هر لایه نقشِ مشخصی در برنامه ایفا می‌کند (مانندِ منطق ارائه یا منطقِ کسب‌وکار). هرچند هیچ محدودیتِ مشخصی از نظرِ شمار و نوعِ لایه‌هایی که باید وجود داشته باشند در کار نیست، بیشترِ معماری‌های لایه‌ای از چهار لایهٔ استاندارد تشکیل می‌شوند: `Presentation`، `Business`، `Persistence` و `Database`. برخی معماری‌ها لایه‌های `Business` و `Persistence` را ترکیب می‌کنند؛ به‌ویژه زمانی که منطقِ ماندگاری (مانندِ `SQL` یا `HSQL`) درونِ مؤلفه‌های لایهٔ `Business` تعبیه شده باشد. برنامه‌های کوچک‌تر ممکن است فقط سه لایه داشته باشند، در حالی که برنامه‌های کسب‌وکاریِ بزرگ‌تر و پیچیده‌تر ممکن است پنج لایه یا بیشتر داشته باشند.



شکل ۱۰-۱. لایه‌های منطقیِ استاندارد در سبک معماری لایه‌ای

شکل ۱۰-۲ گونه‌های توپولوژی را از منظرِ لایه‌بندیِ فیزیکی `(physical layering)` (استقرار) نشان می‌دهد. گونهٔ نخست لایه‌های `Presentation`، `Business` و `Persistence` را در یک واحد استقرار ترکیب می‌کند و لایهٔ `Database` معمولاً به‌صورتِ یک پایگاه‌دادهٔ فیزیکیِ بیرونیِ جداگانه (یا سیستم فایل `(filesystem)`) نمایش داده می‌شود. گونهٔ دوم لایهٔ `Presentation` را به‌طورِ فیزیکی در واحد استقرارِ خودش جدا می‌کند و لایه‌های `Business` و `Persistence` را در واحد استقرارِ دومی ترکیب می‌کند. در این گونه نیز لایهٔ `Database` معمولاً از راهِ یک پایگاه‌داده یا سیستم فایلِ بیرونی به‌طورِ فیزیکی جدا می‌شود. گونهٔ سوم هر چهار لایهٔ استاندارد، از جمله لایهٔ `Database`، را در یک استقرارِ واحد ترکیب می‌کند. این گونه ممکن است برای برنامه‌های کوچک‌تری که پایگاه‌دادهٔ تعبیه‌شدهٔ درونی یا پایگاه‌دادهٔ درون‌حافظه‌ای `(in-memory database)` دارند، مانندِ برنامه‌های دستگاه‌های همراه، سودمند باشد. بسیاری از محصولاتِ درون‌سازمانی (on-premises یا on-prem) با همین گونهٔ سوم ساخته و به مشتریان تحویل داده می‌شوند.



شکل ۱۰-۲. گونه‌های توپولوژیِ فیزیکی (استقرار)

هر لایه نقش و مسئولیتِ مشخصی دارد و پیرامونِ کاری که برای برآوردنِ یک درخواستِ کسب‌وکاریِ خاص باید انجام شود، انتزاعی پدید می‌آورد. برای نمونه، لایهٔ `Presentation` مسئولِ رسیدگی به همهٔ منطقِ رابط کاربری و ارتباط با مرورگر است، در حالی که لایهٔ `Business` مسئولِ اجرای قواعدِ کسب‌وکاریِ مشخصِ مرتبط با درخواست است. لایهٔ `Presentation` لازم نیست بداند یا نگرانِ این باشد که داده‌های مشتری را چگونه به دست آورد؛ فقط باید آن اطلاعات را با قالبی مشخص روی صفحه نمایش دهد. به همین ترتیب، لایهٔ `Business` لازم نیست نگرانِ این باشد که داده‌های مشتری را برای نمایش روی صفحه چگونه قالب‌بندی کند یا حتی آن داده‌ها از کجا می‌آیند؛ فقط باید داده‌ها را از لایهٔ `Persistence` بگیرد، منطقِ کسب‌وکار را روی آن‌ها اجرا کند (مانندِ محاسبهٔ مقادیر یا تجمیعِ داده‌ها) و آن اطلاعات را به لایهٔ `Presentation` بفرستد.

این جداسازیِ دغدغه‌ها (separation of concerns) در سبک معماری لایه‌ای ساختنِ مدل‌های اثربخشِ نقش و مسئولیت را آسان می‌کند. دامنهٔ مؤلفه‌های درونِ هر لایه محدود است و فقط با منطقِ مربوط به همان لایه سروکار دارند. برای نمونه، مؤلفه‌های لایهٔ Presentation فقط منطق ارائه را مدیریت می‌کنند، در حالی که مؤلفه‌های لایهٔ Business فقط منطقِ کسب‌وکار را. این امر به توسعه‌دهندگان امکان می‌دهد از تخصصِ فنیِ خاصِ خود بهره بگیرند و بر جنبه‌های فنیِ حوزه (مانندِ منطق ارائه یا منطقِ ماندگاری) تمرکز کنند. اما بهای این مزیت، نبودِ چابکیِ کل‌نگر (holistic agility) است؛ یعنی توانِ کلِ سیستم برای پاسخ‌گوییِ سریع به تغییر.

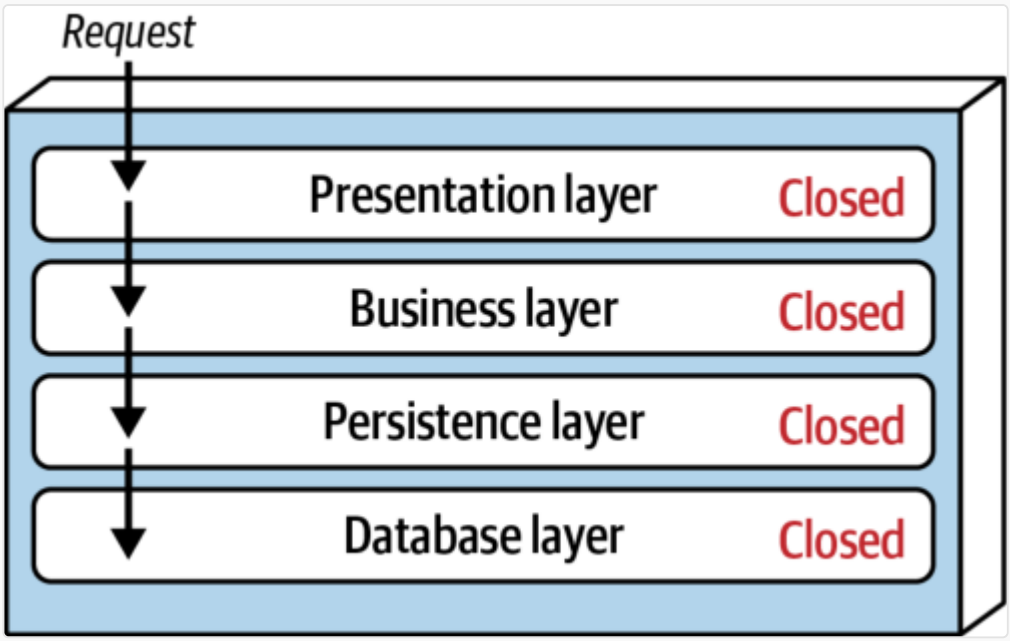
معماری لایه‌ای معماری‌ای افرازشدهٔ فنی است (در برابرِ معماریِ افرازشدهٔ حوزه‌ای). این یعنی، چنان‌که در فصل ۹ آموختید، مؤلفه‌ها بر پایهٔ نقشِ فنی‌شان در معماری (مانندِ ارائه یا کسب‌وکار) از هم جدا می‌شوند، نه بر پایهٔ حوزه (مانندِ مشتری). در نتیجه، هر حوزهٔ کسب‌وکاریِ مشخص در سراسرِ لایه‌های معماری پخش می‌شود. برای نمونه، حوزهٔ «مشتری» در لایه‌های Presentation ، Business ، Rules ، Services و Database جای دارد و همین اعمالِ تغییرات در این حوزه را دشوار می‌کند. در نتیجه، رویکرد DDD با سبک معماری لایه‌ای چندان سازگار نیست.

ویژگی‌های خاصِ سبک

لایه‌ها در این سبک معماری حوزه‌های مشخصی از مسئولیتِ فنی را در خود کپسوله می‌کنند، اما خودِ لایه‌ها ممکن است ویژگی‌های دیگری نیز از خود نشان دهند.

لایه‌های جداسازی

هر لایه می‌تواند بسته یا باز باشد. اگر لایه‌ای بسته (closed) باشد، درخواستی که از لایهٔ بالایی به سوی لایهٔ پایینی حرکت می‌کند نمی‌تواند هیچ لایه‌ای را دور بزند؛ بلکه باید از لایهٔ بلافاصله زیرین عبور کند تا به لایهٔ بعدی برسد (شکل ۳-۱۰ را ببینید). برای نمونه، در معماری‌ای که همهٔ لایه‌هایش بسته‌اند، درخواستی که از لایهٔ Presentation آغاز می‌شود باید نخست از لایهٔ Business و سپس از لایهٔ Persistence بگذرد تا سرانجام به لایهٔ Database برسد.



شکل ۱۰-۳. لایه‌های بسته در یک معماری لایه‌ای

توجه کنید که در شکل ۱۰-۳، برای درخواست‌های بازیابیِ ساده بسیار سریع‌تر و آسان‌تر بود که لایهٔ `Presentation` مستقیماً به پایگاه‌داده دسترسی داشته باشد و لایه‌های غیرضروری را دور بزند (چیزی که در اوایلِ دههٔ ۲۰۰۰ به الگوی `Fast-Lane Reader` شهرت داشت). برای اینکه چنین شود، لایه‌های `Business` و `Persistence` باید باز `(open)` باشند تا درخواست‌ها بتوانند لایه‌ها را دور بزنند. کدام بهتر است، لایه‌های باز یا لایه‌های بسته؟ پاسخِ این پرسش در مفهومی کلیدی به نامِ لایه‌های جداسازی `(layers of isolation)` نهفته است.

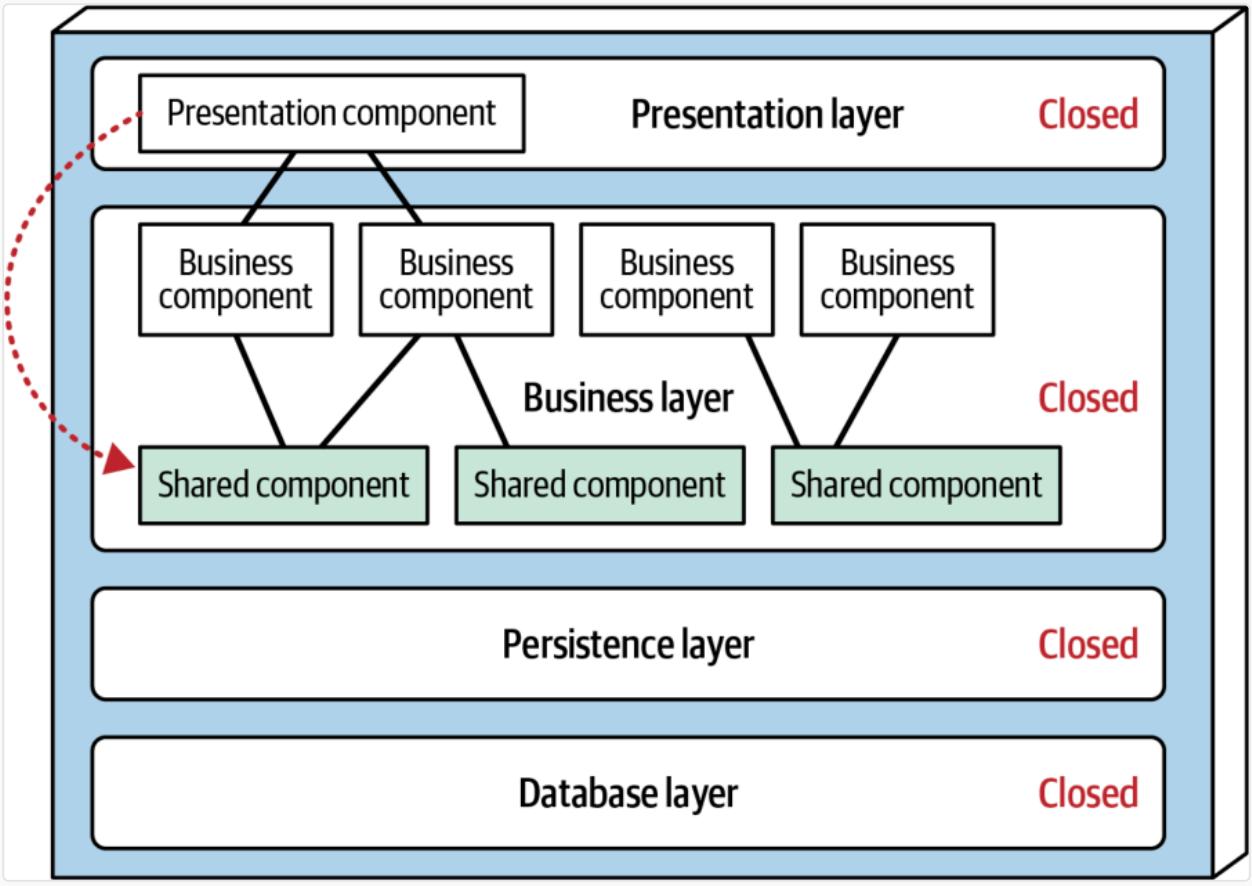
مفهومِ لایه‌های جداسازی یعنی تغییراتی که در یک لایه از معماری اعمال می‌شوند، به‌طورِ کلی بر مؤلفه‌های لایه‌های دیگر اثر نمی‌گذارند؛ به شرطِ آنکه قراردادهای میانِ آن لایه‌ها بدونِ تغییر بمانند. هر لایه از لایه‌های دیگر مستقل است و از سازوکارِ درونیِ آن‌ها شناختِ اندکی دارد یا هیچ شناختی ندارد. اما برای پشتیبانی از لایه‌های جداسازی، لایه‌هایی که در جریانِ اصلیِ یک درخواست درگیرند باید بسته باشند. اگر لایهٔ `Presentation` بتواند مستقیماً به لایهٔ `Persistence` دسترسی داشته باشد، تغییرات در لایهٔ `Persistence` هم بر لایهٔ `Business` و هم بر لایهٔ `Presentation` اثر می‌گذارد و برنامه‌ای با کوپلینگ بسیار تنگاتنگ و وابستگی‌های متقابلِ لایه‌ای میانِ مؤلفه‌ها پدید می‌آورد. این امر معماری لایه‌ای را بسیار شکننده و تغییرش را دشوار و پرهزینه می‌کند.

به‌کارگیریِ لایه‌های جداسازی همچنین امکان می‌دهد هر لایه‌ای از معماری را بدونِ اثرگذاری بر لایه‌های دیگر جایگزین کرد (باز هم با فرضِ قراردادهای خوش‌تعریف و به‌کارگیریِ الگوی `Business Delegate`) [پانویس ۱]. برای نمونه، می‌توانید با بهره‌گیری از لایه‌های جداسازی، چارچوبِ رابط کاربریِ قدیمی‌تان را با چارچوبی جدیدتر جایگزین کنید و تمامیِ این تغییرات را تنها در لایهٔ `Presentation` اعمال نمایید.

پانویس ۱ – `Business Delegate` الگویی است که برای کاهشِ کوپلینگ میانِ سرویس‌های کسب‌وکاری و رابط کاربری طراحی شده است. `Business Delegate`ها نقشِ مبدل `(adapter)` را دارند و اشیای کسب‌وکاری را از لایهٔ ارائه فراخوانی می‌کنند.

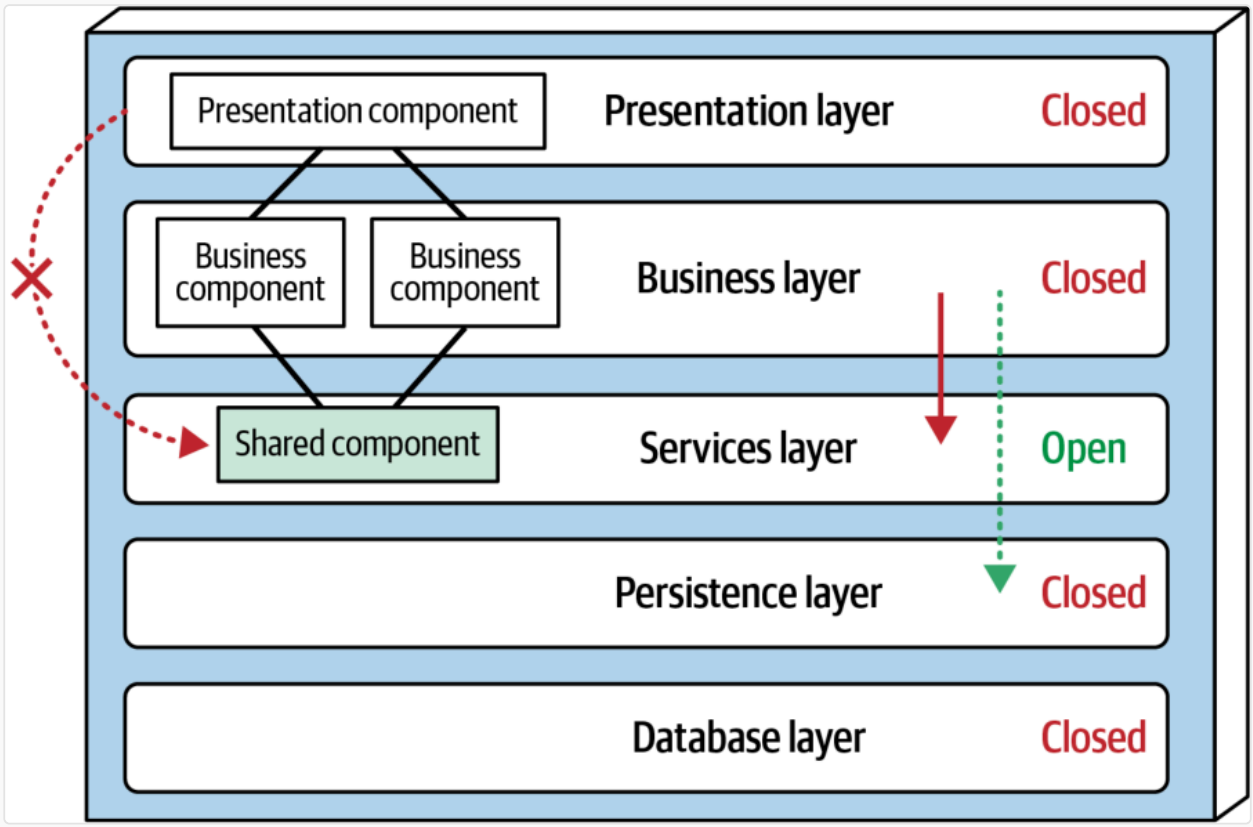
افزودنِ لایه‌ها

هرچند لایه‌های بسته لایه‌های جداسازی را ممکن می‌کنند و از این راه به مهارِ تغییر کمک می‌کنند، گاهی منطقی است که برخی لایه‌ها باز باشند. برای نمونه، فرض کنید لایهٔ `Business` در معماری لایه‌ای شما اشیای مشترکی `(shared objects)` دارد که کارکردهای مشترکِ مؤلفه‌های کسب‌وکاری را در خود جای داده‌اند (مانندِ کلاس‌های کمکیِ تاریخ و رشته، کلاس‌های ممیزی، کلاس‌های ثبت وقایع و مانندِ آن‌ها). شما تصمیمی معماری می‌گیرید که استفادهٔ لایهٔ `Presentation` از این اشیای کسب‌وکاریِ مشترک را محدود می‌کند. این قید در شکل ۱۰-۴ با خطِ نقطه‌چین نشان داده شده است که از یک مؤلفهٔ ارائه به یک شیء کسب‌وکاریِ مشترک در لایهٔ `Business` می‌رود. حاکمیت بر این سناریو و کنترلِ آن دشوار است، زیرا از نظرِ معماری، لایهٔ `Presentation` به لایهٔ `Business` و در نتیجه به اشیای مشترکِ درونِ آن لایه دسترسی دارد.



شکل ۱۰-۴. اشیای مشترک در لایهٔ `Business`

یکی از راه‌های اعمالِ این محدودیت در سطحِ معماری، افزودنِ یک لایهٔ `Services` جدید است که همهٔ اشیای کسب‌وکاریِ مشترک را در خود جای دهد (شکل ۱۰-۵ را ببینید). افزودنِ این لایهٔ جدید، از نظرِ معماری، دسترسیِ لایهٔ `Presentation` به اشیای کسب‌وکاریِ مشترک را محدود می‌کند، زیرا لایهٔ `Business` بسته است. اما باید لایهٔ `Services` جدید را باز تعریف کنید؛ در غیرِ این صورت، لایهٔ `Business` ناچار می‌شود برای دسترسی به لایهٔ `Persistence` از لایهٔ `Services` بگذرد. باز تعریف کردنِ لایهٔ `Services` به لایهٔ `Business` امکان می‌دهد یا به آن لایه دسترسی پیدا کند (چنان‌که پیکانِ توپُر نشان می‌دهد) یا آن را دور بزند و به لایهٔ زیرینِ بعدی برود (چنان‌که پیکانِ نقطه‌چین نشان می‌دهد).



شکل ۱۰-۵. افزودنِ یک لایهٔ `Services` جدید به معماری

بهره‌گیری از مفهومِ لایه‌های باز و بسته به تعریفِ رابطهٔ میانِ لایه‌های معماری و جریان‌های درخواست `(request flows)` کمک می‌کند. این مفهوم همچنین اطلاعات و راهنماییِ لازم را در اختیارِ توسعه‌دهندگان می‌گذارد تا محدودیت‌های دسترسی به لایه‌ها را در معماری درک کنند. مستندنکردن یا انتقالِ نادرستِ اینکه کدام لایه‌های معماری باز و کدام بسته‌اند (و چرا)، معمولاً به معماری‌هایی شکننده با کوپلینگ تنگاتنگ می‌انجامد که آزمودن، نگه‌داری و استقرارشان بسیار دشوار است.

توپولوژی‌های داده

معماری‌های لایه‌ای به‌طورِ سنتی سیستمی تک‌پارچه را در کنارِ یک پایگاه‌دادهٔ واحد و تک‌پارچه شکل می‌دهند. لایهٔ `Persistence` مشترک اغلب برای نگاشتِ سلسله‌مراتبِ اشیا میانِ زبان‌های شیءگرای محبوب و قلمروِ مجموعه‌محورِ پایگاه‌داده‌های رابطه‌ای به کار می‌رود.

ملاحظات ابری

از آنجا که معماری‌های لایه‌ای معمولاً تک‌پارچه‌اند و به لایه‌ها افراز شده‌اند، گزینه‌های ابری به استقرارِ یک یا چند لایه از راهِ یک ارائه‌دهندهٔ ابر محدود می‌شود. افرازِ فنیِ ذاتیِ این معماری برای استقرارهای جداگانه از راهِ ابر بسیار مناسب است. اما تأخیرِ ارتباط میانِ سرورهای درون‌سازمانی و ابر ممکن است مشکل‌ساز شود، زیرا گردش‌کارها در این معماری معمولاً از بیشترِ لایه‌ها می‌گذرند.

ریسک‌های رایج

معماری‌های لایه‌ای، به دلیلِ استقرارهای تک‌پارچه و نبودِ ماژولاریتیِ معماری، از تحمل خطا پشتیبانی نمی‌کنند. اگر بخشِ کوچکی از یک معماری لایه‌ای دچارِ خطای کمبودِ حافظه `(out-of-memory)` شود، کلِ واحدِ برنامه از کار می‌افتد. در دسترس‌بودنِ کلی نیز به دلیلِ میانگین زمانِ بازیابی (mean time to recover یا `MTTR`) بالای بیشترِ برنامه‌های تک‌پارچه آسیب می‌بیند: زمانِ راه‌اندازی از ۲ دقیقه در برنامه‌های کوچک‌تر تا ۱۵ دقیقه و حتی بیشتر در اکثرِ برنامه‌های بزرگ متغیر است.

حاکمیت

وضعیتِ حاکمیت در این سبک معماری بسیار مطلوب است: از آنجا که این سبک بسیار رایج است، معمارانی که برخی از نخستین ابزارهای آزمونِ ساختاری `(structural testing tools)` را ساختند، این معماری را در نظر داشتند. در واقع، تابع برازشِ نمونه در شکل ۶-۴ برای معماری لایه‌ای ساخته شده بود (مثال ۱۰-۱ را ببینید).

مثال ۱۰-۱. تابع برازشِ `ArchUnit` برای حاکمیت بر لایه‌ها

```
layeredArchitecture()
.layer("Controller").definedBy("..controller..")
.layer("Service").definedBy("..service..")
.layer("Persistence").definedBy("..persistence..")

.whereLayer("Controller").mayNotBeAccessedByAnyLayer()
.whereLayer("Service").mayOnlyBeAccessedByLayers("Controller")
.whereLayer("Persistence").mayOnlyBeAccessedByLayers("Service")
```

در مثال ۱۰-۱، معمار لایه‌های معماری را تعریف می‌کند و نام‌هایی گویا، مانندِ `Controller`، برای مؤلفه‌ای که با نامِ بسته بازنمایی می‌شود، تعیین می‌کند. (در نحوِ `ArchUnit`، دو نقطه `..` در دو سوی نامِ یک بسته نشان‌دهندهٔ مالکیتِ آن است.) سپس معمار ارتباطِ مجاز میانِ لایه‌ها را تعریف می‌کند و بدین ترتیب بر لایه‌های باز و بستهٔ آن‌ها حاکمیت می‌کند.

کتابخانه‌های توابع برازش از سبک معماری لایه‌ای به‌خوبی پشتیبانی می‌کنند و به معماران امکان می‌دهند حاکمیت بر روابطی را که میانِ لایه‌ها طراحی کرده‌اند، در طولِ پیاده‌سازی خودکار کنند.

هر معماری لایه‌ای دست‌کم چند سناریو دارد که در دامِ ضدالگوی چاهکِ معماری `(Architecture Sinkhole)` می‌افتند. این ضدالگو زمانی رخ می‌دهد که درخواست‌ها بدونِ اجرای هیچ منطقِ کسب‌وکاری صرفاً از لایه‌ای به لایهٔ دیگر دست‌به‌دست شوند. برای نمونه، فرض کنید لایهٔ `Presentation` به درخواستِ سادهٔ کاربری برای بازیابیِ داده‌های پایه‌ای مشتری (مانندِ نام و نشانی) پاسخ می‌دهد. لایهٔ `Presentation` درخواست را به لایهٔ `Business` می‌دهد، که کاری جز سپردنِ آن به لایهٔ `Rules` نمی‌کند؛ لایهٔ `Rules` نیز کاری جز سپردنِ آن به لایهٔ `Persistence` نمی‌کند و لایهٔ `Persistence` سپس با یک فراخوانیِ سادهٔ `SQL` داده‌های مشتری را از لایهٔ `Database` بازیابی می‌کند. آنگاه داده‌ها بی‌هیچ پردازش یا منطقِ افزوده‌ای برای تجمیع، محاسبه، اعمالِ قواعد یا تبدیل، تمامِ مسیرِ پشته را به بالا بازمی‌گردند. نتیجه، نمونه‌سازی و پردازشِ غیرضروریِ اشیاست که هم حافظه را هدر می‌دهد و هم کارایی را کاهش می‌دهد.

کلیدِ تشخیصِ اینکه آیا این ضدالگو در کار است یا نه، تحلیلِ درصدِ درخواست‌هایی است که در این دسته قرار می‌گیرند. قاعدهٔ ۸۰-۲۰ `(80-20 rule)` معمولاً معیارِ مناسبی برای ارزیابی است. برای نمونه، اگر فقط ۲۰٪ درخواست‌ها چاهک باشند، پذیرفتنی است؛ اما اگر این رقم ۸۰٪ باشد، نشانهٔ خوبی است که معماری لایه‌ای سبک معماریِ درستی برای حوزهٔ مسئله نیست. رویکردِ دیگر برای حلِ ضدالگوی چاهکِ معماری، باز کردنِ همهٔ لایه‌های معماری است؛ البته با آگاهی از اینکه بهای این کار، دشوارترشدنِ مدیریتِ تغییر است.

ملاحظاتِ توپولوژی تیم

برخلافِ برخی از سبک‌های معماریِ شرح‌داده‌شده در این کتاب، سبک معماری لایه‌ای به‌طورِ کلی مستقل از توپولوژی‌های تیم است و با هر پیکربندیِ تیمی کار می‌کند:

تیم‌های هم‌راستا با جریان: از آنجا که معماری لایه‌ای معمولاً کوچک و خودبسنده است و یک سفر یا جریانِ واحد را در سیستم بازنمایی می‌کند، با تیم‌های هم‌راستا با جریان به‌خوبی کار می‌کند. در این توپولوژیِ تیم، تیم‌ها معمولاً مالکِ جریان در سیستم، از آغاز تا پایان و در گذر از هر لایه، هستند و گردش‌کارها را به‌عنوانِ بخشی از راه‌حل‌هایشان می‌سازند.

تیم‌های توانمندساز: از آنجا که معماری لایه‌ای بسیار ماژولار است و بر پایهٔ دغدغه‌های فنی جدا شده است، به‌خوبی با توپولوژی‌های تیمِ توانمندساز جفت می‌شود. متخصصان و اعضای میان‌بُرشیِ تیم `(cross-cutting team members)` می‌توانند با یک یا چند لایه تعامل کنند تا پیشنهادهایی بدهند و آزمایش‌هایی انجام دهند، بی‌آنکه بر

بقیهٔ جریان اثر بگذارند. برای نمونه، تیم می‌تواند با افزودنِ رفتارهای جدید به لایهٔ `Presentation`، یک کتابخانهٔ رابط کاربریِ جدید را بیازماید و هم‌زمان لایه‌های دیگر را از این تغییرات دور نگه دارد.

تیم‌های زیرسیستم پیچیده: از آنجا که هر لایه کارِ بسیار مشخصی انجام می‌دهد، این سبک با توپولوژیِ تیمِ زیرسیستم پیچیده به‌خوبی کار می‌کند. برای نمونه، لایهٔ `Persistence` نقطهٔ اتصالِ بی‌نقصی برای تیمی فراهم می‌کند که برای اهدافِ تحلیلی به داده‌های عملیاتی `(operational data)` دسترسی نیاز دارد. اگر به تیمِ زیرسیستم پیچیده اجازهٔ دسترسی در سطحِ لایهٔ `Persistence` داده شود، این تیم می‌تواند بی‌آنکه بر لایه‌های دیگری که همچنان در مالکیتِ تیم‌های هم‌راستا با جریان‌اند اثر بگذارد، کار کند.

تیم‌های پلتفرم: تیم‌های پلتفرمی که روی یک معماری لایه‌ای کار می‌کنند، می‌توانند با به‌کارگیریِ ابزارهای فراوانی که برای آن در دسترس است، از درجهٔ بالای ماژولاریتیِ آن بهره ببرند.

بزرگ‌ترین چالشِ معماری‌های لایه‌ای برای بیشترِ تیم‌های پلتفرم همان مشکلِ کلیِ تک‌پارچه‌هاست: هرچه بزرگ‌تر می‌شوند، دست‌وپاگیرتر می‌شوند. اگر تیم‌ها پیوسته قابلیت‌های تازه به یک تک‌پارچه بیفزایند، هر اندازه هم که خوب افراز شده باشد و حاکمیت بر آن به‌خوبی اعمال شود، سرانجام زیرِ فشارِ برخی قیود قرار می‌گیرد: اتصال‌های پایگاه‌داده، حافظه، کارایی، کاربران هم‌زمان یا انبوهی از مشکلاتِ قریب‌الوقوعِ دیگر. عملیاتی نگه‌داشتنِ سیستم، تیمِ پلتفرم را به انجامِ کارهایی روزبه‌روز دشوارتر وادار خواهد کرد.

ویژگی‌های سبک

در جدول رتبه‌بندیِ ویژگی‌ها `(characteristics ratings)` که در شکل ۱۰-۶ آمده است، رتبهٔ یک‌ستاره یعنی آن ویژگی معماریِ مشخص در این معماری به‌خوبی پشتیبانی نمی‌شود، در حالی که رتبهٔ پنج‌ستاره یعنی آن ویژگی معماری یکی از قوی‌ترین جنبه‌های این سبک معماری است. تعریفِ هر یک از ویژگی‌هایی که در این کارنامه `(scorecard)` آمده‌اند در فصل ۴ یافت می‌شود.

	Architectural characteristic	Star rating
	Overall cost	$
Structural	Partitioning type	Technical
	Number of quanta	1
	Simplicity	★★★★★
	Modularity	★
Engineering	Maintainability	★
	Testability	★★
	Deployability	★
	Evolvability	★
Operational	Responsiveness	★★★
	Scalability	★
	Elasticity	★
	Fault tolerance	★

شکل ۱۰-۶. رتبه‌بندیِ ویژگی‌های معماری لایه‌ای

هزینهٔ کلی `(overall cost)` و سادگی نقاطِ قوتِ اصلیِ سبک معماری لایه‌ای‌اند. معماری‌های لایه‌ای، از آنجا که تک‌پارچه‌اند، به پیچیدگیِ سبک‌های معماری توزیع‌شده نیستند؛ ساده‌ترند، فهمشان آسان‌تر است و ساخت و نگه‌داری‌شان نسبتاً کم‌هزینه است. اما احتیاط کنید، زیرا این رتبه‌ها با بزرگ‌ترشدنِ معماری لایه‌ایِ تک‌پارچه، و در نتیجه پیچیده‌ترشدنِ آن، به‌سرعت افت می‌کنند.

نه استقرارپذیری و نه آزمون‌پذیری در این سبک معماری رتبهٔ خوبی ندارند. استقرارپذیری پایین است، زیرا استقرارها پرریسک و کم‌تکرارند و تشریفات `(ceremony)` و تلاشِ فراوانی می‌طلبند. برای نمونه، برای اعمالِ یک تغییرِ سادهٔ سه‌خطی در یک فایلِ کلاس، باید کلِ واحد استقرار دوباره مستقر شود؛ و همین احتمالِ آن را پدید می‌آورد که تغییراتی

در پایگاه‌داده، پیکربندی یا جنبه‌های دیگرِ کد نیز در کنارِ تغییرِ اصلی پنهانی راه یابند. افزون بر این، همین تغییرِ سادهٔ سه‌خطی معمولاً با ده‌ها تغییرِ دیگر یک‌جا بسته‌بندی می‌شود که هر یک از آن‌ها ریسکِ استقرار را افزایش داده و بسامدِ استقرارها را کمتر می‌کند. رتبهٔ پایینِ آزمون‌پذیری نیز بازتابِ همین سناریوست: بیشترِ توسعه‌دهندگان برای یک تغییرِ سادهٔ سه‌خطی ساعت‌ها وقت صرفِ اجرای کلِ مجموعهٔ آزمونِ رگرسیون (regression test suite) نمی‌کنند (با این فرض که اصلاً چنین مجموعه‌آزمونی داشته باشند). ما به آزمون‌پذیری رتبهٔ دوستاره (و نه یک‌ستاره) می‌دهیم، زیرا این سبک امکانِ ماک کردن (mock) یا استاب کردن (stub) مؤلفه‌ها یا حتی یک لایهٔ کامل را فراهم می‌کند و همین تلاشِ کلیِ آزمون را آسان‌تر می‌کند.

ویژگی‌های مهندسی (engineering characteristics) سبک معماری لایه‌ای بازتابِ همان پویایی‌ای است که در بالا گفته شد: همه در آغاز خوب‌اند، اما با رشدِ اندازهٔ پایگاه کد افت می‌کنند.

کشسانی و مقیاس‌پذیری در معماری لایه‌ای رتبهٔ بسیار پایینی (یک‌ستاره) دارند که دلیلِ اصلی‌اش استقرارهای تک‌پارچه و نبودِ ماژولاریتیِ معماری است. هرچند می‌توان برخی کارکردهای درونِ یک تک‌پارچه را بیش از کارکردهای دیگر مقیاس‌پذیر کرد، این تلاش معمولاً به فنونِ طراحیِ بسیار پیچیده‌ای نیاز دارد که این معماری برایشان مناسب نیست؛ مانندِ چندریسمانی (multithreading)، پیام‌رسانیِ داخلی (internal messaging) و دیگر رویه‌های پردازش موازی (parallel processing). با این حال، از آنجا که کوانتوم معماریِ یک سیستمِ لایه‌ای همیشه ۱ است (به دلیلِ رابط کاربری و پایگاه‌دادهٔ تک‌پارچه و پردازشِ سمتِ سرورش)، برنامه‌ها فقط تا حدِ معینی مقیاس‌پذیرند.

معماران می‌توانند با طراحیِ دقیق در معماری لایه‌ای به پاسخ‌گوییِ (responsiveness) بالایی دست یابند و با فنونی مانندِ کش‌گذاری و چندریسمانی آن را بیشتر هم کنند. ما در مجموع به این سبک سه ستاره می‌دهیم، زیرا از نبودِ پردازش موازیِ ذاتی، و همچنین از لایه‌بندیِ بسته و ضدالگوی چاهک معماری، آسیب می‌بیند.

چه زمانی استفاده کنیم

سبک معماری لایه‌ای گزینهٔ خوبی برای برنامه‌ها یا وب‌سایت‌های کوچک و ساده است. این سبک همچنین نقطهٔ شروعِ خوبی برای شرایطی با محدودیت‌های شدیدِ بودجه و زمان است. این سبک، به دلیلِ سادگی و آشنا بودنش برای توسعه‌دهندگان و معماران، شاید یکی از کم‌هزینه‌ترین سبک‌ها باشد و توسعهٔ برنامه‌های کوچک‌تر را آسان می‌کند. سبک معماری لایه‌ای همچنین گزینهٔ خوبی است وقتی معماران هنوز در حالِ ارزیابیِ این موضوع هستند که آیا معماری‌های پیچیده‌تر مناسب‌ترند یا نه، اما باید فرایندِ توسعه را آغاز کنند.

هنگامِ به‌کارگیریِ این روش، بازاستفادهٔ کد را در کمترین حد و سلسله‌مراتبِ اشیا، یعنی عمقِ درختِ وراثت (inheritance tree)، را نسبتاً کم‌عمق نگه دارید تا سطحِ خوبی از ماژولاریتی حفظ شود. این کار مهاجرت به سبک معماریِ دیگری را در آینده آسان‌تر می‌کند.

چه زمانی استفاده نکنیم

چنان‌که نشان دادیم، ویژگی‌هایی مانندِ قابلیت نگهداری، چابکی، آزمون‌پذیری و استقرارپذیری با رشدِ برنامه‌هایی که از سبک معماری لایه‌ای استفاده می‌کنند، آسیب می‌بینند. به همین دلیل، برای برنامه‌ها و سیستم‌های بزرگ شاید بهتر باشد از سبک‌های معماریِ دیگر و ماژولارتر استفاده شود.

نمونه‌ها و موارد کاربرد

معماری لایه‌ای یکی از رایج‌ترین سبک‌های معماری است و در زمینه‌های فراوانی پدیدار می‌شود.

طراحانِ سیستم‌عامل‌ها (مانندِ `Linux` یا `Windows`) به همان دلیلی از لایه‌ها استفاده می‌کنند که معمارانِ برنامه‌ها: جداسازی دغدغه‌ها. لایه‌های رایج در سیستم‌عامل‌ها عبارت‌اند از:

لایهٔ سخت‌افزار **`(Hardware layer)`**: سخت‌افزارِ فیزیکی، مانندِ `CPU`، حافظه و دستگاه‌های ورودی/خروجی `(I/O)`، را در بر می‌گیرد.

لایهٔ کرنل **`(Kernel layer)`**: انتزاعِ سخت‌افزار، مدیریتِ حافظه و زمان‌بندیِ فرایندها را فراهم می‌کند.

لایهٔ واسطِ فراخوانیِ سیستمی **`(System Call Interface layer)`**: با کرنل تعامل می‌کند تا سرویس‌های سیستمی را فراهم کند.

لایهٔ کاربر **`(User layer)`**: برنامه‌ها و ابزارهای کاربردی‌ای را در بر می‌گیرد که کاربران با آن‌ها تعامل دارند.

مدلِ شبکه‌ایِ اتصالِ متقابلِ سیستم‌های باز (Open Systems Interconnection یا OSI) چگونگیِ مرزبندیِ مسئولیت در شبکه‌ها را ترسیم می‌کند. برای نمونه، پروتکلِ پایهٔ اینترنت، `TCP/IP`، لایه‌های زیر را در بر می‌گیرد:

لایهٔ فیزیکی **`(Physical layer)`**: داده‌ها را به‌طورِ فیزیکی انتقال می‌دهد.

لایهٔ پیوند داده **`(Data Link layer)`**: از تشخیصِ خطا و همگام‌سازیِ قاب `(frame synchronization)` بهره می‌گیرد.

لایهٔ شبکه **`(Network layer)`**: مسیریابی `(routing)` را انجام می‌دهد (مانندِ `IP`).

لایهٔ انتقال **`(Transport layer)`**: انتقالِ مطمئنِ داده‌ها را تضمین می‌کند (مانندِ `TCP`).

لایهٔ کاربرد **`(Application layer)`**: سرویس‌هایی مانندِ ایمیل `(SMTP)`، انتقالِ فایل `(FTP)` و مرورِ وب `(HTTP)` را فراهم می‌کند.

معماری لایه‌ای جداسازی دغدغه‌ها را ترویج می‌کند، قابلیت نگهداری را بهبود می‌دهد و امکانِ توسعهٔ مستقلِ هر لایه را فراهم می‌کند؛ بنابراین هر معماری‌ای که برای این ویژگی‌ها ارزش قائل است، از آن سود خواهد برد.

معماری لایه‌ای همچنین برای تیم‌هایی که می‌کوشند به ویژگی معماریِ امکان‌پذیری دست یابند بسیار رایج است: آیا واقعاً می‌توانیم دامنهٔ اعلام‌شده را در زمانِ تعیین‌شده و با منابعِ در دسترس تحویل دهیم؟ برای نمونه، اگر سازمانی با سرمایهٔ سرمایه‌گذاران پیش می‌رود و باید هرچه سریع‌تر چیزی تحویل دهد، سادگیِ معماری لایه‌ای اغلب آن را به گزینهٔ خوبی تبدیل می‌کند؛ حتی اگر بعدها لازم باشد بخش‌هایی از آن برای دستیابی به قابلیت‌های دیگر بازنویسی شوند.

واژه‌نامهٔ اختصاصی این فصل

واژهٔ انگلیسی	معادل مصوب/به‌کاررفته در این فصل
Layered Architecture	معماری لایه‌ای
N-Tiered Architecture	معماری چندلایه
Architecture by Implication (antipattern)	معماری ضمنی
Accidental Architecture (antipattern)	معماری اتفاقی
Physical Layering	لایه‌بندی فیزیکی
Filesystem	سیستم فایل
In-Memory Database	پایگاه‌دادهٔ درون‌حافظه‌ای
On-Premises (On-Prem)	درون‌سازمانی
Separation of Concerns	جداسازی دغدغه‌ها
Holistic Agility	چابکی کل‌نگر
Closed Layer	لایهٔ بسته
Open Layer	لایهٔ باز
Layers of Isolation	لایه‌های جداسازی
Fast-Lane Reader (pattern)	الگوی Fast-Lane Reader (بدون ترجمه)
Business Delegate (pattern)	الگوی Business Delegate (بدون ترجمه)
Adapter	مبدل
Shared Object	شیء مشترک
Request Flow	جریان درخواست

واژهٔ انگلیسی	معادل مصوب/به‌کاررفته در این فصل
Out-of-Memory	کمبود حافظه
Mean Time to Recover (MTTR)	میانگین زمانِ بازیابی
Structural Testing Tools	ابزارهای آزمون ساختاری
Architecture Sinkhole (antipattern)	چاهک معماری
80-20 Rule	قاعدهٔ ۸۰-۲۰
Cross-Cutting Team Members	اعضای میان‌بُرشی تیم
Operational Data	دادهٔ عملیاتی
Fault Tolerance	تحمل خطا
Availability	در دسترس‌بودن
Concurrent Users	کاربران هم‌زمان
Characteristics Ratings	رتبه‌بندی ویژگی‌ها
Scorecard	کارنامه
Overall Cost	هزینهٔ کلی
Ceremony (deployment)	تشریفات
Regression Test Suite	مجموعهٔ آزمون رگرسیون
Mock	ماک
Stub	استاب
Engineering Characteristics	ویژگی‌های مهندسی
Multithreading	چندریسمانی
Internal Messaging	پیام‌رسانی داخلی
Parallel Processing	پردازش موازی
Responsiveness	پاسخ‌گویی
Inheritance Tree	درخت وراثت

واژهٔ انگلیسی	معادل مصوب/به‌کاررفته در این فصل
Hardware Layer	لایهٔ سخت‌افزار
Kernel Layer	لایهٔ کرنل
System Call Interface Layer	لایهٔ واسط فراخوانی سیستمی
User Layer	لایهٔ کاربر
Open Systems Interconnection (OSI) Model	مدل اتصال متقابل سیستم‌های باز (OSI)
Physical Layer	لایهٔ فیزیکی
Data Link Layer	لایهٔ پیوند داده
Network Layer	لایهٔ شبکه
Transport Layer	لایهٔ انتقال
Application Layer	لایهٔ کاربرد
Frame Synchronization	همگام‌سازی قاب
Routing	مسیریابی